

# National University of Computing & Emerging Sciences



**AL2002 – Artificial Intelligence – Lab (Spring 2026)**

**BSCS-6C**

**Lab 16 (Multilayer Perceptron)**

|                       |                         |
|-----------------------|-------------------------|
| <b>Lab Instructor</b> | <b>Muzaffar Abbas</b>   |
| <b>Department</b>     | <b>Computer Science</b> |

# Multi-Layer Perceptron Learning

*Feed Forward + Backpropagation*

## Question1:

**Assumptions:**  $\theta_5$ ,  $\theta_6$ , and  $\theta_7$  are treated as bias values. Target  $t = 1$  and learning rate  $\eta = 0.1$ .

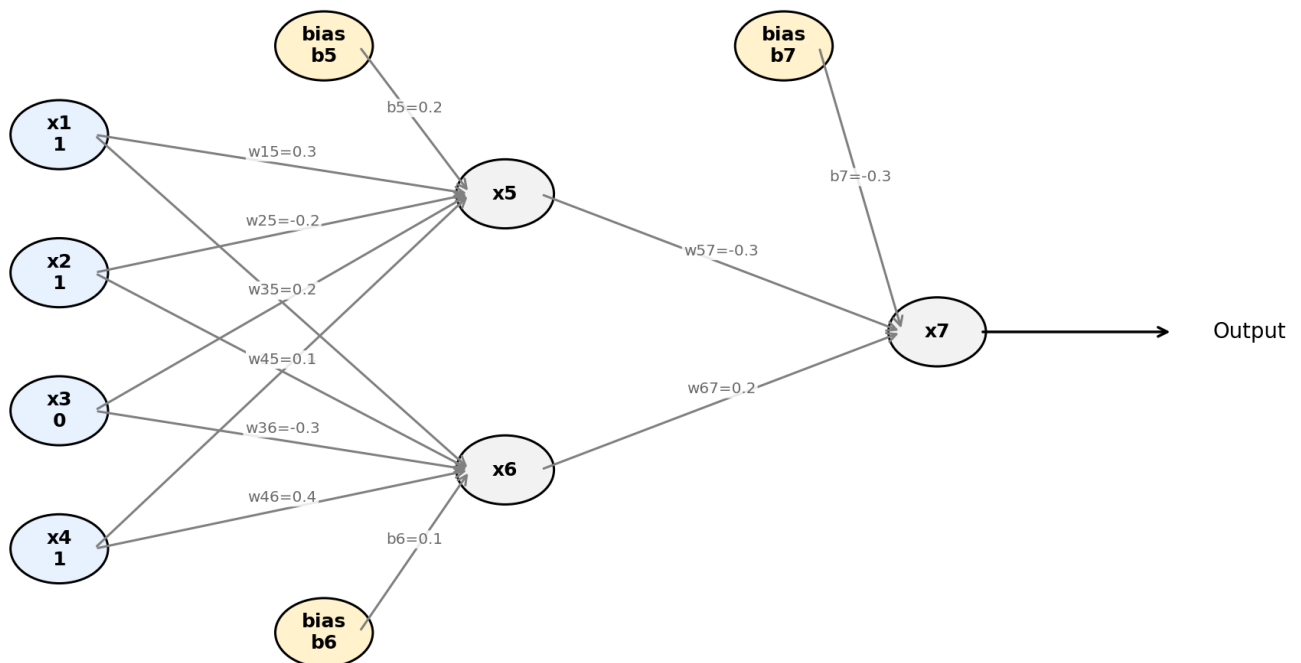
Activation function:  $\text{sigmoid}(\text{net}) = 1 / (1 + e^{-(\text{net})})$

Weight update rule:  $\text{new weight} = \text{old weight} + \eta \times \delta \times \text{input}$

## Given Values

| Inputs                       | Input $\rightarrow$ Hidden Weights                                                                             | Hidden $\rightarrow$ Output Weights | Biases                                                          |
|------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------------|
| $x_1=1, x_2=1, x_3=0, x_4=1$ | $w_{15}=0.3, w_{16}=0.1$<br>$w_{25}=-0.2, w_{26}=0.4$<br>$w_{35}=0.2, w_{36}=-0.3$<br>$w_{45}=0.1, w_{46}=0.4$ | $w_{57}=-0.3$<br>$w_{67}=0.2$       | $b_5=\theta_5=0.2$<br>$b_6=\theta_6=0.1$<br>$b_7=\theta_7=-0.3$ |

**Step 0: Initial MLP with Given Inputs, Weights, and Biases**



# 1. Feed Forward Pass

## 1.1 Calculate Hidden Neuron x5

$$\text{net}_5 = x_1w_{15} + x_2w_{25} + x_3w_{35} + x_4w_{45} + b_5$$

$$\text{net}_5 = 1(0.3) + 1(-0.2) + 0(0.2) + 1(0.1) + 0.2 = 0.4$$

$$x_5 = \text{sigmoid}(0.4) = 0.5987$$

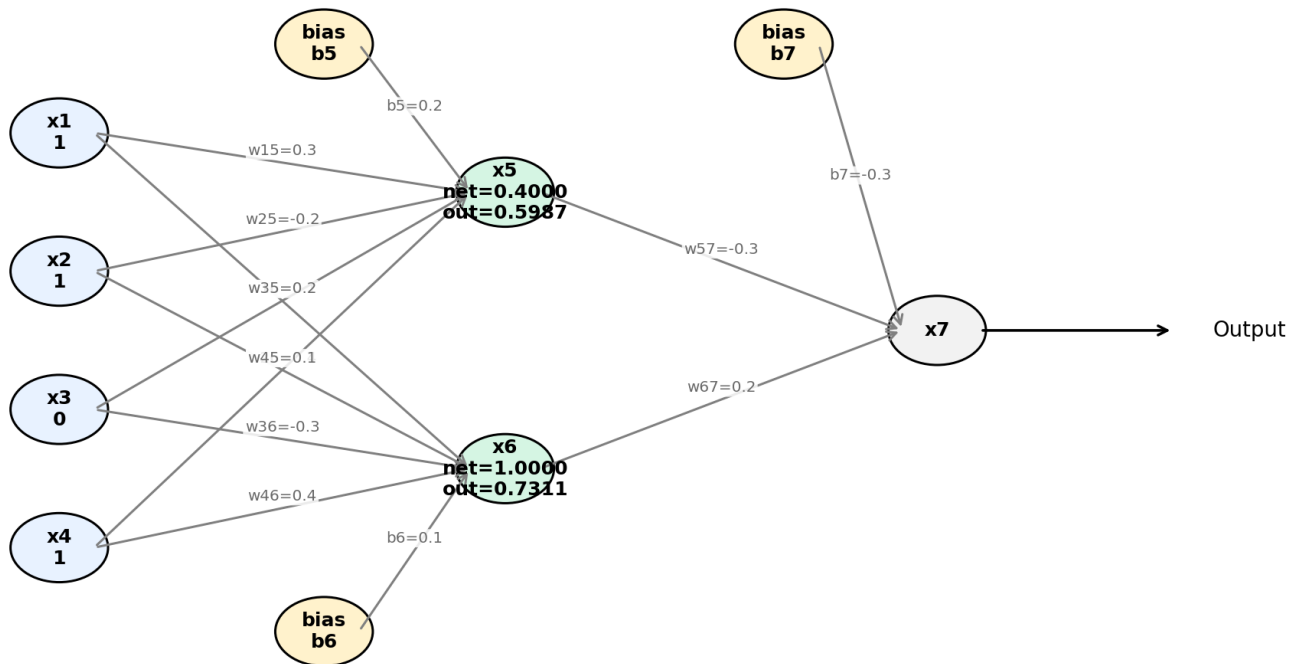
## 1.2 Calculate Hidden Neuron x6

$$\text{net}_6 = x_1w_{16} + x_2w_{26} + x_3w_{36} + x_4w_{46} + b_6$$

$$\text{net}_6 = 1(0.1) + 1(0.4) + 0(-0.3) + 1(0.4) + 0.1 = 1.0$$

$$x_6 = \text{sigmoid}(1.0) = 0.7311$$

### Step 1: Hidden Layer Values Calculated



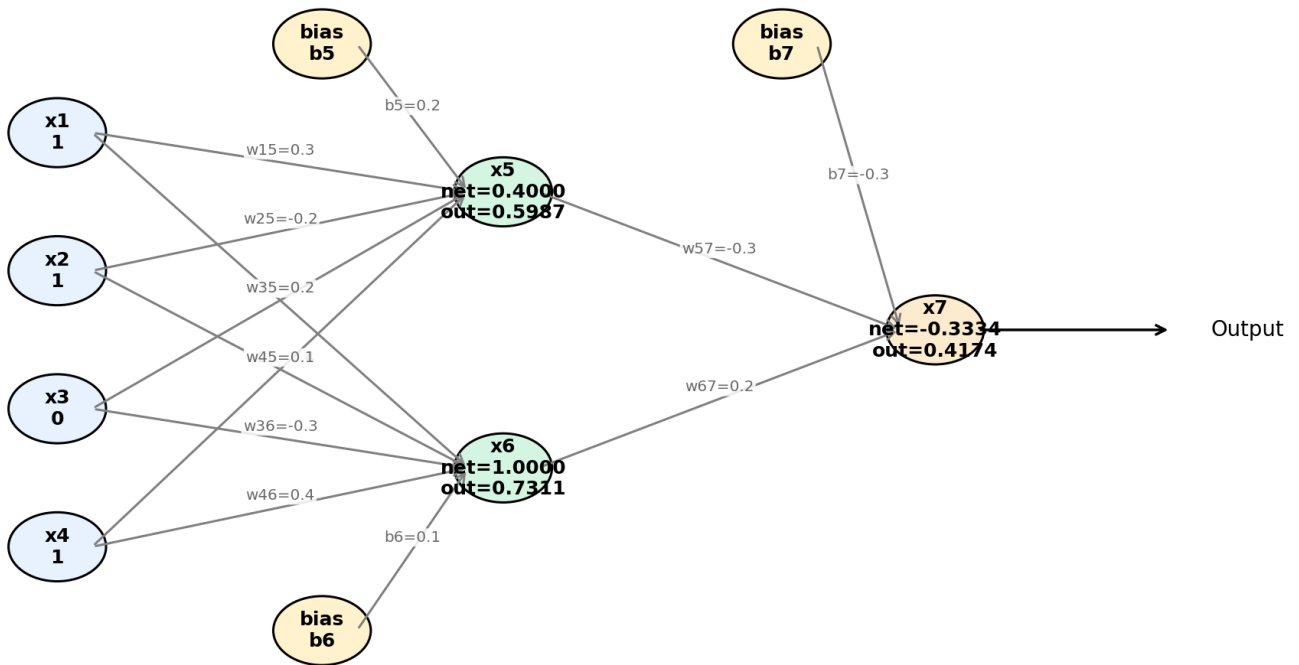
## 1.3 Calculate Output Neuron x7

$$\text{net}_7 = x_5w_{57} + x_6w_{67} + b_7$$

$$\text{net}_7 = 0.5987(-0.3) + 0.7311(0.2) + (-0.3) = -0.3334$$

$$\text{output} = x_7 = \text{sigmoid}(-0.3334) = 0.4174$$

## Step 2: Output Layer Value Calculated



## 2. Error Calculation

Raw error = target - actual = 1 - 0.4174 = 0.5826

Squared error/loss =  $1/2(\text{target} - \text{actual})^2 = 1/2(0.5826)^2 = 0.1697$

For backpropagation, the raw error target - actual is used inside the delta formula.

## 3. Backpropagation

### 3.1 Output Layer Delta

$$\delta_7 = \text{output}(1 - \text{output})(\text{target} - \text{output})$$

$$\delta_7 = 0.4174(1 - 0.4174)(1 - 0.4174) = 0.1417$$

### 3.2 Hidden Layer Deltas

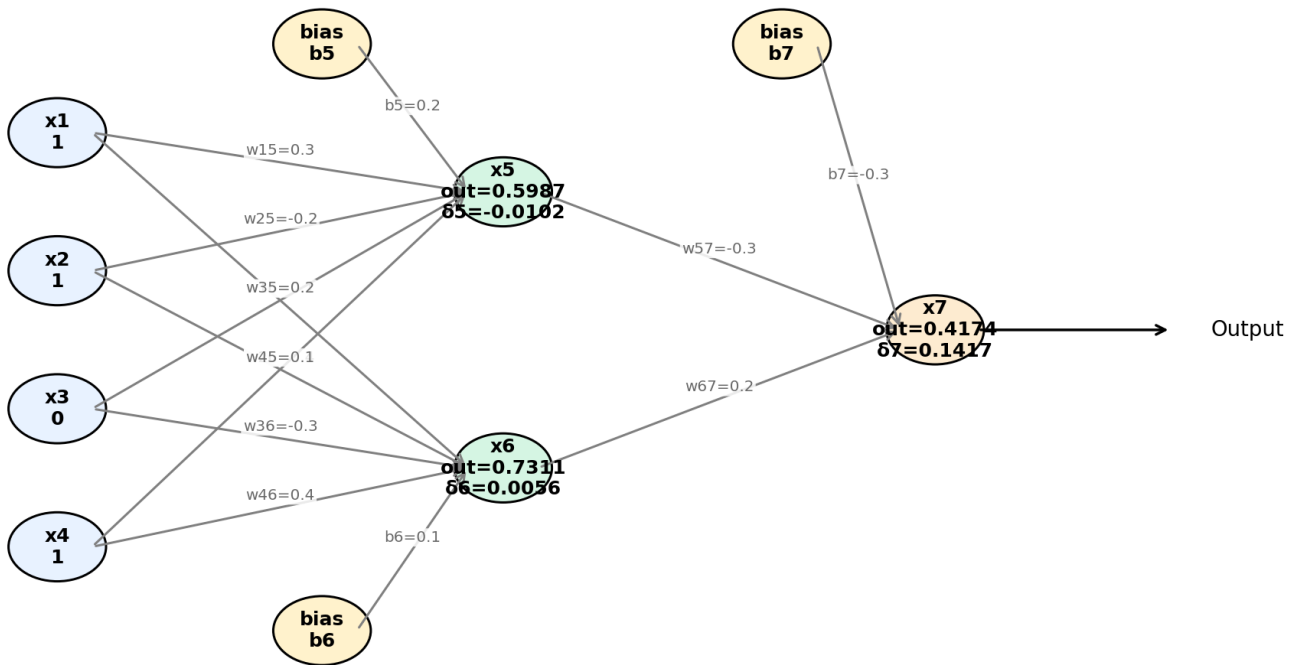
$$\delta_5 = x_5(1 - x_5)(\delta_7 \times w_{57})$$

$$\delta_5 = 0.5987(1 - 0.5987)(0.1417 \times -0.3) = -0.0102$$

$$\delta_6 = x_6(1 - x_6)(\delta_7 \times w_{67})$$

$$\delta_6 = 0.7311(1 - 0.7311)(0.1417 \times 0.2) = 0.0056$$

### Step 3: Backpropagation Deltas Calculated

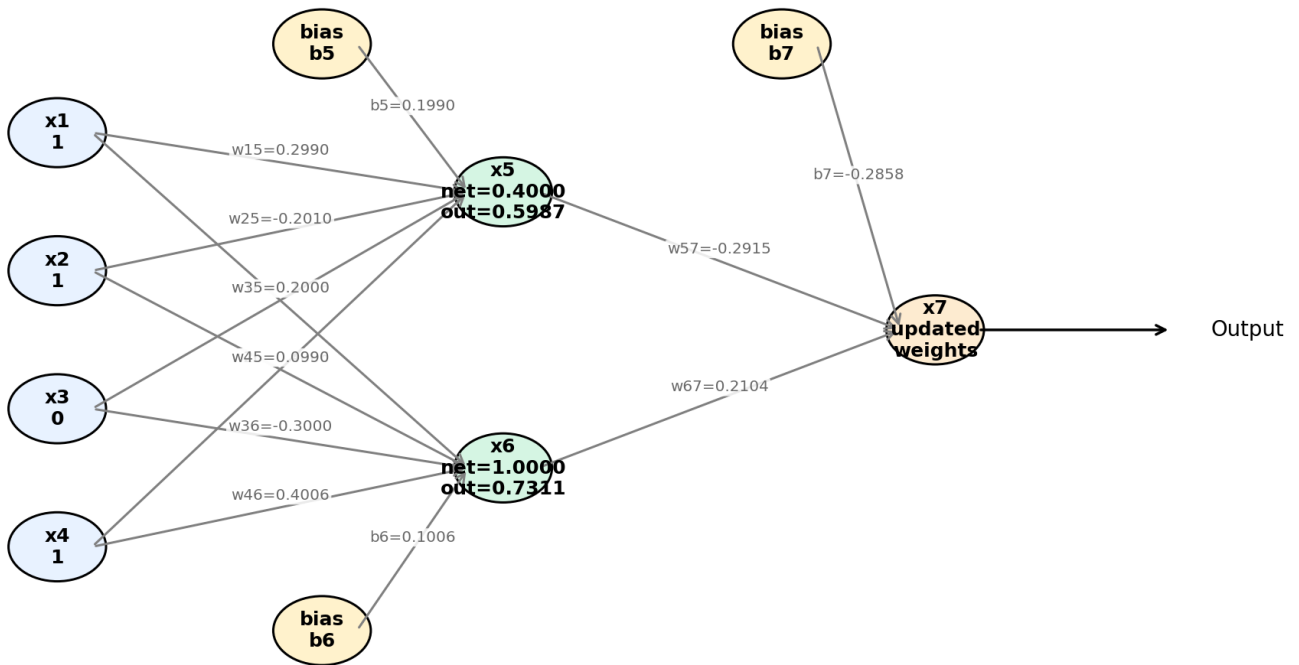


## 4. Update Weights and Biases

Formula: new value = old value +  $\eta \times \delta \times \text{input}$

| Value | Old     | Change  | New     |
|-------|---------|---------|---------|
| w15   | 0.3000  | -0.0010 | 0.2990  |
| w25   | -0.2000 | -0.0010 | -0.2010 |
| w35   | 0.2000  | -0.0000 | 0.2000  |
| w45   | 0.1000  | -0.0010 | 0.0990  |
| b5    | 0.2000  | -0.0010 | 0.1990  |
| w16   | 0.1000  | +0.0006 | 0.1006  |
| w26   | 0.4000  | +0.0006 | 0.4006  |
| w36   | -0.3000 | +0.0000 | -0.3000 |
| w46   | 0.4000  | +0.0006 | 0.4006  |
| b6    | 0.1000  | +0.0006 | 0.1006  |
| w57   | -0.3000 | +0.0085 | -0.2915 |
| w67   | 0.2000  | +0.0104 | 0.2104  |
| b7    | -0.3000 | +0.0142 | -0.2858 |

## Step 4: Network After Weight and Bias Updates



## Final Summary

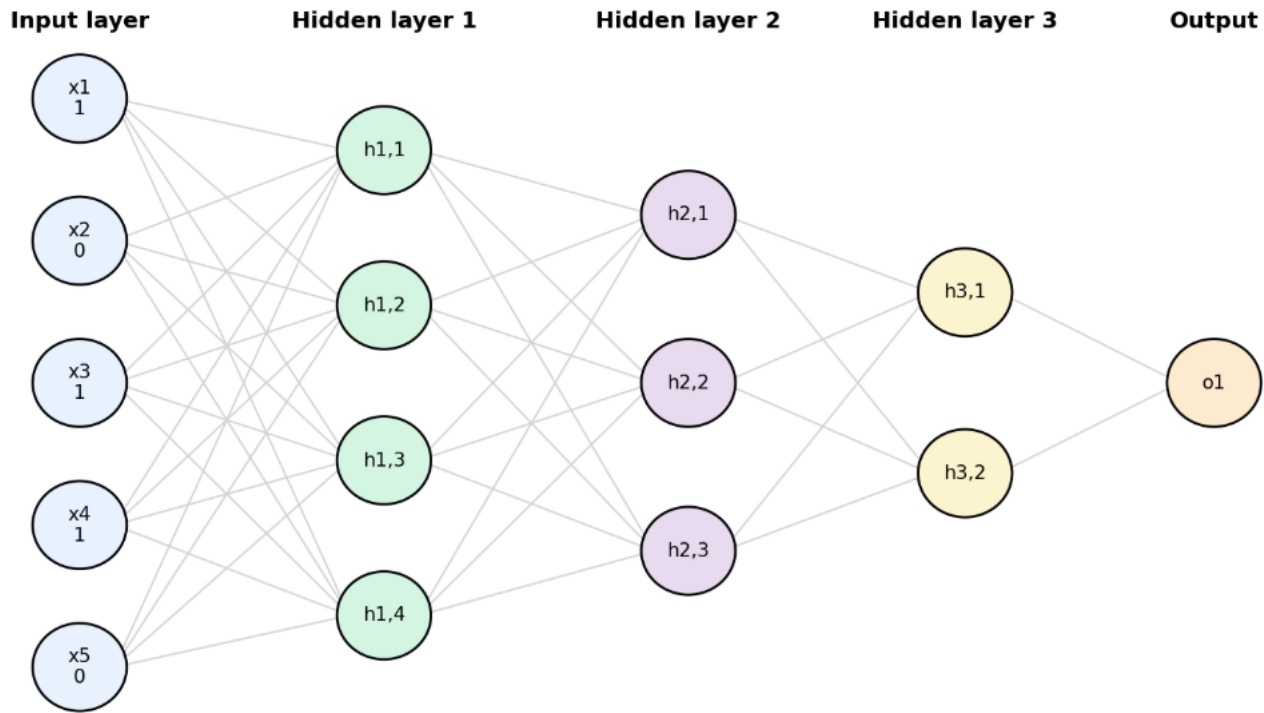
- Hidden output  $x_5 = 0.5987$
- Hidden output  $x_6 = 0.7311$
- Final output  $x_7 = 0.4174$
- Raw error = 0.5826
- Loss = 0.1697
- $\delta_7 = 0.1417$ ,  $\delta_5 = -0.0102$ ,  $\delta_6 = 0.0056$
- After backpropagation, you again perform feedforward and keep repeating the cycle until the network learns
- So, what is the stopping Criteria
- keep repeating until **one of these conditions is met:**
- Error becomes very small (**most common**)
- Maximum iterations reached (e.g., **after 1000 epochs**)
- No significant improvement (**Weights stop changing much**)
- But from coding point of view use the following:
- repeat until:
  - error < small value OR max epochs reached
- Set the **small value** to **0.01** and **max epochs** value to be **1000**

## Question 2: Multi-Layer Perceptron Training

Architecture: 5 Inputs  $\rightarrow$  4 Hidden  $\rightarrow$  3 Hidden  $\rightarrow$  2 Hidden  $\rightarrow$  1 Output

Assumed values: target  $t = 1$ , learning rate  $\eta = 0.1$ , activation =  $\text{sigmoid}(\text{net}) = 1 / (1 + e^{(-\text{net})})$ .

### Step 0: Network Structure and Given Input Values



Fully connected layers. Edge labels are kept in tables to avoid clutter.

### 1. Given Values

Input vector:  $x = [1, 0, 1, 1, 0]$

#### Weights: Input $\rightarrow$ Hidden Layer 1

| From / To | h1,1  | h1,2  | h1,3  | h1,4  | Bias |
|-----------|-------|-------|-------|-------|------|
| x1        | 0.20  | -0.10 | 0.40  | 0.10  |      |
| x2        | 0.00  | 0.30  | -0.20 | 0.20  |      |
| x3        | -0.30 | 0.20  | 0.10  | -0.10 |      |
| x4        | 0.40  | 0.10  | 0.00  | 0.30  |      |
| x5        | 0.10  | -0.20 | 0.30  | 0.20  |      |
| Bias      | 0.10  | 0.00  | -0.10 | 0.20  |      |

#### Weights: Hidden Layer 1 $\rightarrow$ Hidden Layer 2

| From / To | h2,1 | h2,2  | h2,3  | Bias |
|-----------|------|-------|-------|------|
| h1,1      | 0.30 | -0.20 | 0.10  |      |
| h1,2      | 0.20 | 0.40  | -0.30 |      |

|      |       |       |       |  |
|------|-------|-------|-------|--|
| h1,3 | -0.10 | 0.10  | 0.20  |  |
| h1,4 | 0.40  | -0.10 | 0.30  |  |
| Bias | 0.00  | 0.10  | -0.10 |  |

### Weights: Hidden Layer 2 → Hidden Layer 3

| From / To | h3,1  | h3,2  | Bias |
|-----------|-------|-------|------|
| h2,1      | 0.20  | -0.30 |      |
| h2,2      | 0.10  | 0.40  |      |
| h2,3      | -0.20 | 0.20  |      |
| Bias      | 0.10  | -0.10 |      |

### Weights: Hidden Layer 3 → Output

| From / To | o1    | Bias |
|-----------|-------|------|
| h3,1      | 0.30  |      |
| h3,2      | -0.20 |      |
| Bias      | 0.00  |      |

## 2. Feed Forward Pass

### 2.1 First Hidden Layer

$net\_h1,1 = 1(0.20) + 0(0.00) + 1(-0.30) + 1(0.40) + 0(0.10) + bias(0.10) = 0.4000$ ;  $h1,1 = \text{sigmoid}(0.4000) = 0.5987$

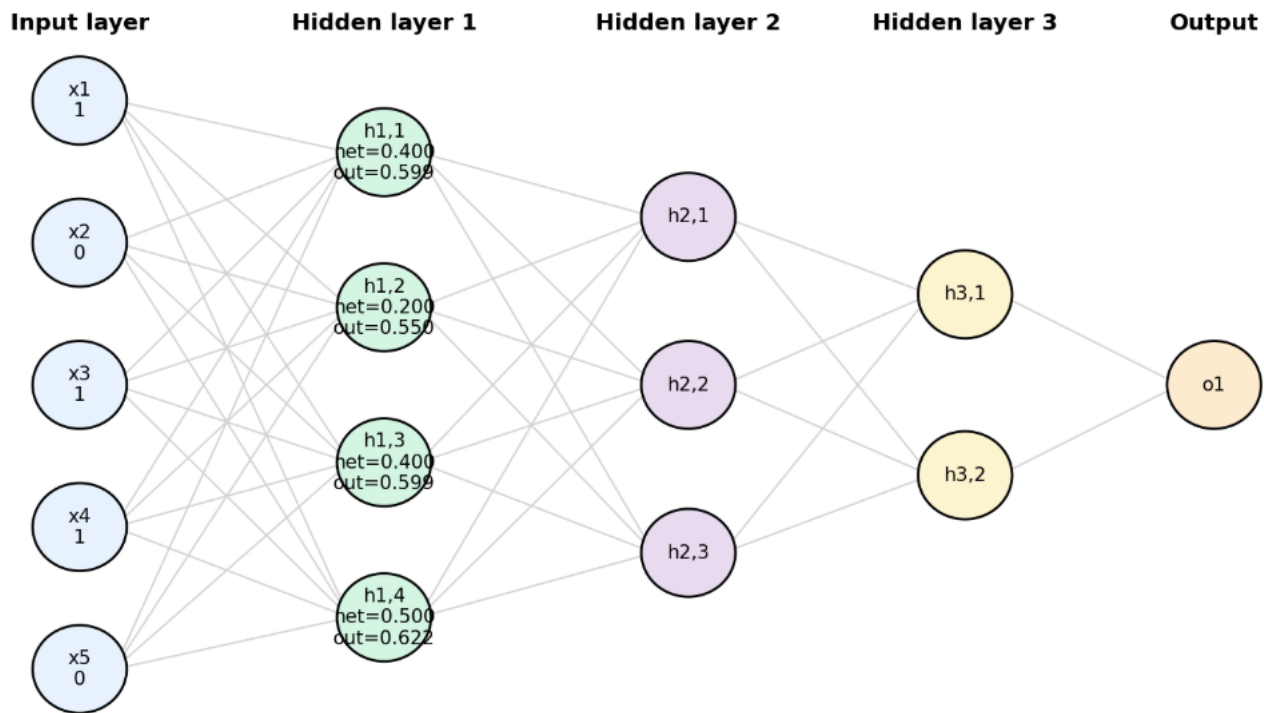
$net\_h1,2 = 1(-0.10) + 0(0.30) + 1(0.20) + 1(0.10) + 0(-0.20) + bias(0.00) = 0.2000$ ;  $h1,2 = \text{sigmoid}(0.2000) = 0.5498$

$net\_h1,3 = 1(0.40) + 0(-0.20) + 1(0.10) + 1(0.00) + 0(0.30) + bias(-0.10) = 0.4000$ ;  $h1,3 = \text{sigmoid}(0.4000) = 0.5987$

$net\_h1,4 = 1(0.10) + 0(0.20) + 1(-0.10) + 1(0.30) + 0(0.20) + bias(0.20) = 0.5000$ ;  $h1,4 = \text{sigmoid}(0.5000) = 0.6225$



## Step 1: First Hidden Layer Calculated



Fully connected layers. Edge labels are kept in tables to avoid clutter.

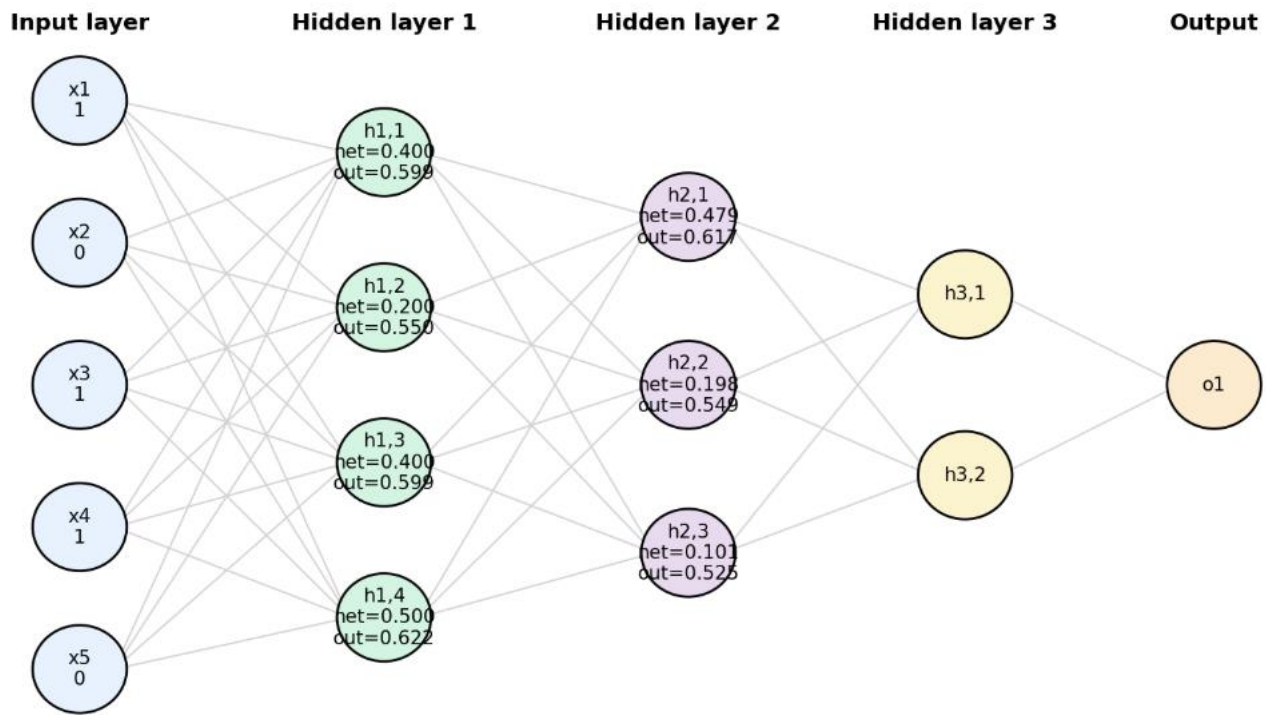
### 2.2 Second Hidden Layer

$\text{net\_}h2,1 = 0.5987(0.30) + 0.5498(0.20) + 0.5987(-0.10) + 0.6225(0.40) + \text{bias}(0.00) = 0.4787$ ;  $h2,1 = 0.6174$

$\text{net\_}h2,2 = 0.5987(-0.20) + 0.5498(0.40) + 0.5987(0.10) + 0.6225(-0.10) + \text{bias}(0.10) = 0.1978$ ;  $h2,2 = 0.5493$

$\text{net\_}h2,3 = 0.5987(0.10) + 0.5498(-0.30) + 0.5987(0.20) + 0.6225(0.30) + \text{bias}(-0.10) = 0.1014$ ;  $h2,3 = 0.5253$

## Step 2: Second Hidden Layer Calculated

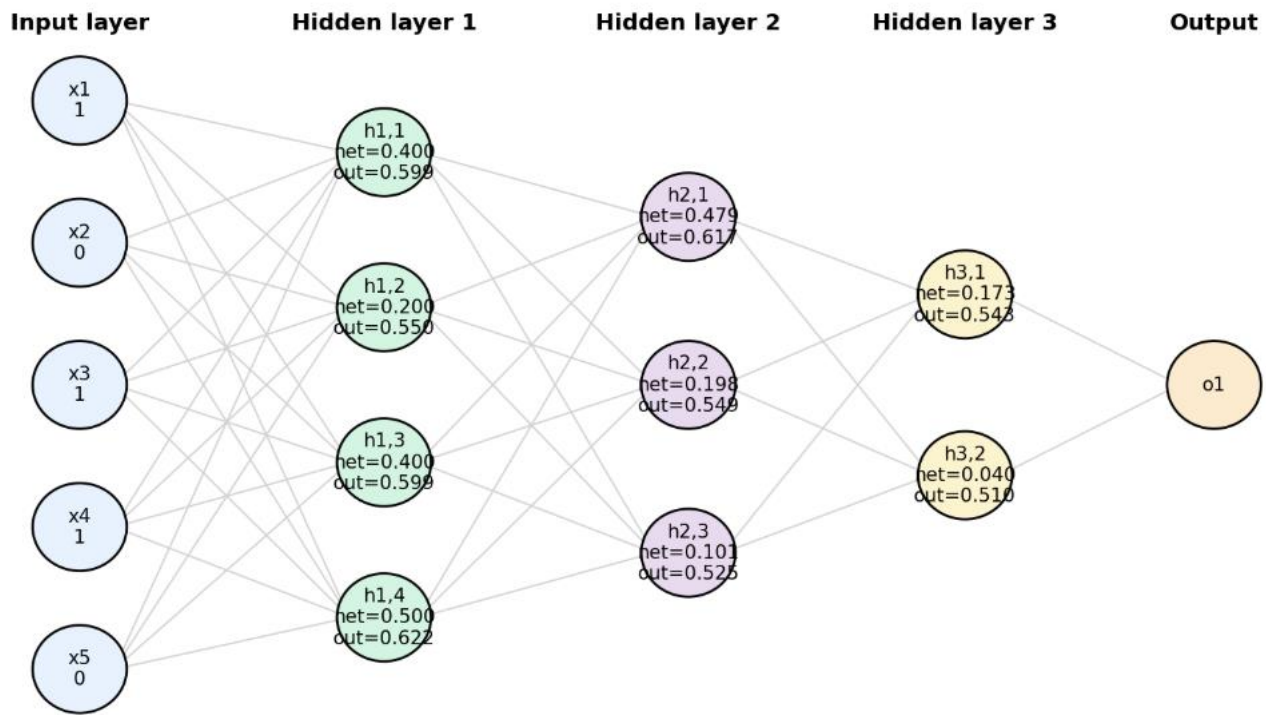


### 2.3 Third Hidden Layer

$\text{net}_{h3,1} = 0.6174(0.20) + 0.5493(0.10) + 0.5253(-0.20) + \text{bias}(0.10) = 0.1734$ ;  $h3,1 = 0.5432$

$\text{net}_{h3,2} = 0.6174(-0.30) + 0.5493(0.40) + 0.5253(0.20) + \text{bias}(-0.10) = 0.0396$ ;  $h3,2 = 0.5099$

### Step 3: Third Hidden Layer Calculated



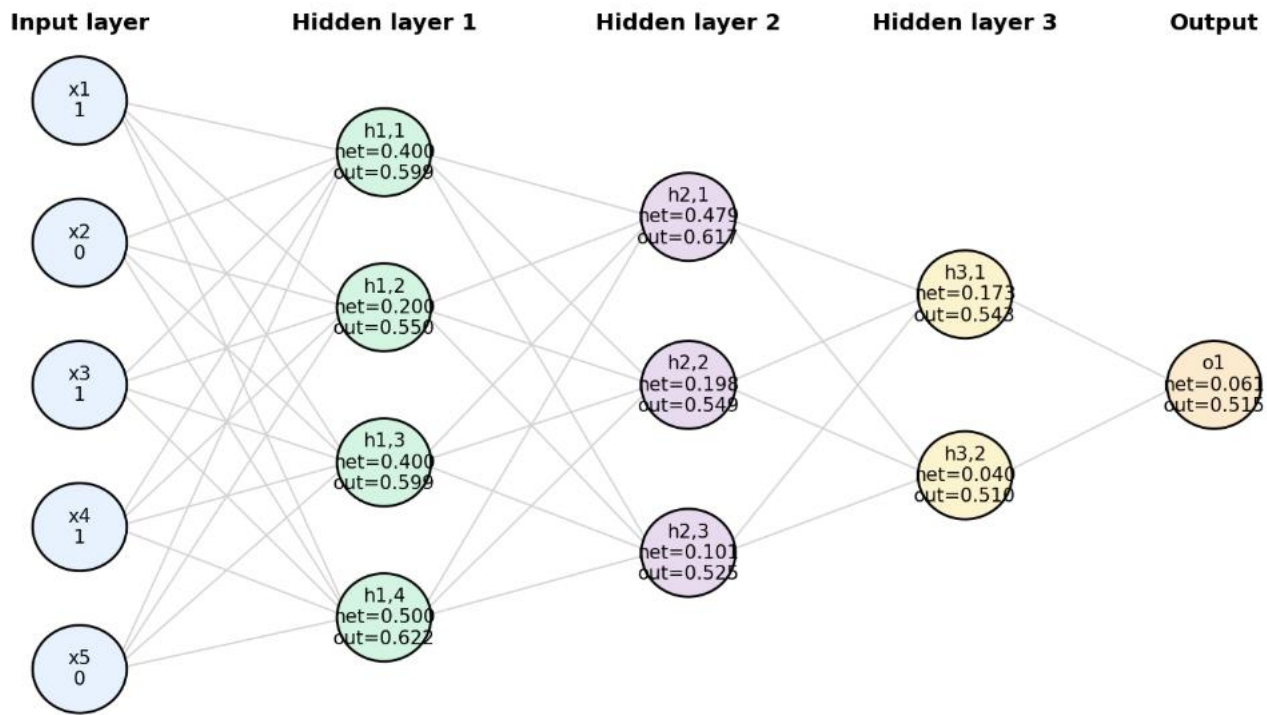
Fully connected layers. Edge labels are kept in tables to avoid clutter.

#### 2.4 Output Layer

$$\text{net\_o1} = 0.5432(0.30) + 0.5099(-0.20) + \text{bias}(0.00) = 0.0610$$

$$\text{Actual output } y = \text{sigmoid}(0.0610) = 0.5152$$

## Step 4: Final Output Calculated



Fully connected layers. Edge labels are kept in tables to avoid clutter.

### 3. Error Calculation

Raw error = target - actual = 1 - 0.5152 = 0.4848

Squared error/loss =  $\frac{1}{2}(\text{target} - \text{actual})^2 = \frac{1}{2}(0.4848)^2 = 0.1175$

### 4. Backpropagation

Output delta formula:  $\delta_{\text{output}} = \text{output}(1 - \text{output})(\text{target} - \text{output})$

$\delta_{o1} = 0.5152(1 - 0.5152)(1 - 0.5152) = 0.1211$

Hidden delta formula:  $\delta_{\text{hidden}} = \text{output}(1 - \text{output}) \times \text{sum}(\text{next layer delta} \times \text{connecting weight})$

$\delta_{h3,1} = 0.0090$

$\delta_{h3,2} = -0.0061$

$\delta_{h2,1} = 0.0009$

$\delta_{h2,2} = -0.0004$

$\delta_{h2,3} = -0.0008$

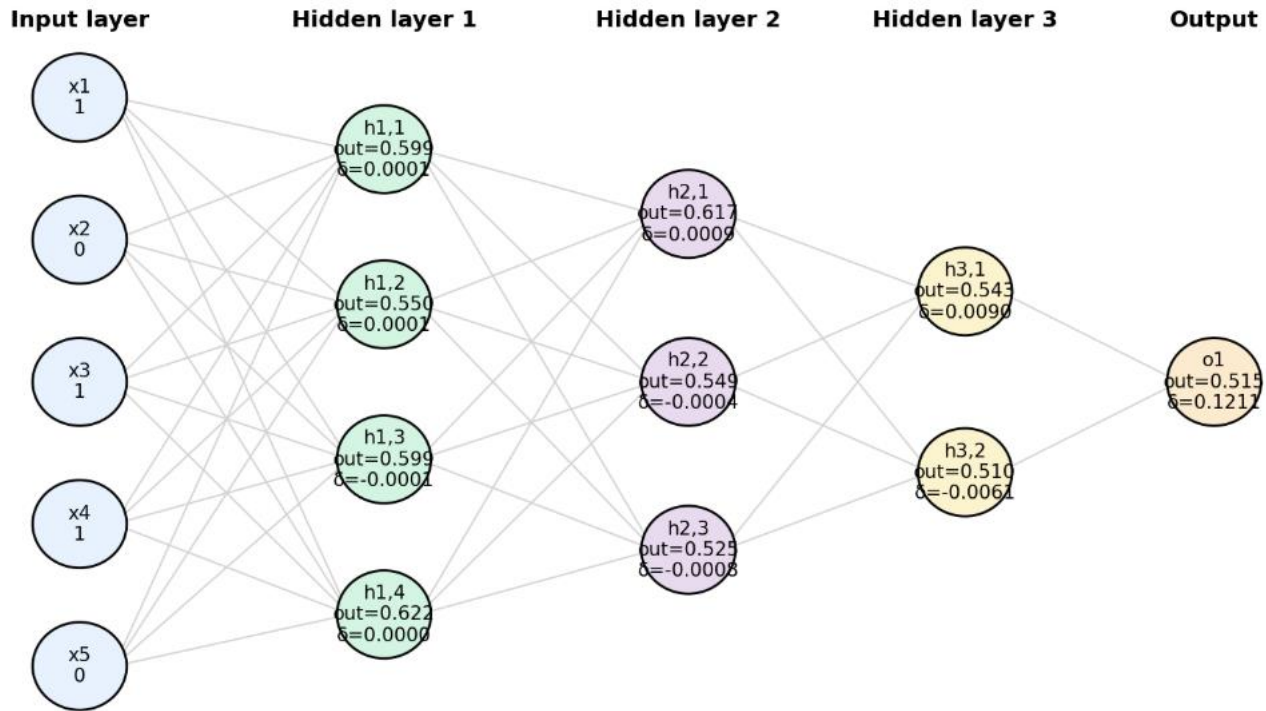
$\delta_{h1,1} = 0.0001$

$\delta_{h1,2} = 0.0001$

$$\delta_{h1,3} = -0.0001$$

$$\delta_{h1,4} = 0.0000$$

### Step 5: Backpropagation Deltas Calculated



Fully connected layers. Edge labels are kept in tables to avoid clutter.

## 5. Weight and Bias Updates

Update rule: new weight = old weight +  $\eta \times \delta \times \text{input\_to\_that\_weight}$ . Bias update: new bias = old bias +  $\eta \times \delta$ .

### Updates: Hidden Layer 3 → Output

| Value                    | Old     | Change  | New     | Note |
|--------------------------|---------|---------|---------|------|
| $w(h3,1 \rightarrow o1)$ | 0.3000  | +0.0066 | 0.3066  |      |
| $w(h3,2 \rightarrow o1)$ | -0.2000 | +0.0062 | -0.1938 |      |
| $b(o1)$                  | 0.0000  | +0.0121 | 0.0121  | bias |

### Updates: Hidden Layer 2 → Hidden Layer 3

| Value                      | Old     | Change  | New     | Note |
|----------------------------|---------|---------|---------|------|
| $w(h2,1 \rightarrow h3,1)$ | 0.2000  | +0.0006 | 0.2006  |      |
| $w(h2,1 \rightarrow h3,2)$ | -0.3000 | -0.0004 | -0.3004 |      |
| $w(h2,2 \rightarrow h3,1)$ | 0.1000  | +0.0005 | 0.1005  |      |
| $w(h2,2 \rightarrow h3,2)$ | 0.4000  | -0.0003 | 0.3997  |      |
| $w(h2,3 \rightarrow h3,1)$ | -0.2000 | +0.0005 | -0.1995 |      |
| $w(h2,3 \rightarrow h3,2)$ | 0.2000  | -0.0003 | 0.1997  |      |
| $b(h3,1)$                  | 0.1000  | +0.0009 | 0.1009  | bias |

|         |         |         |         |      |
|---------|---------|---------|---------|------|
| b(h3,2) | -0.1000 | -0.0006 | -0.1006 | bias |
|---------|---------|---------|---------|------|

### Updates: Hidden Layer 1 → Hidden Layer 2

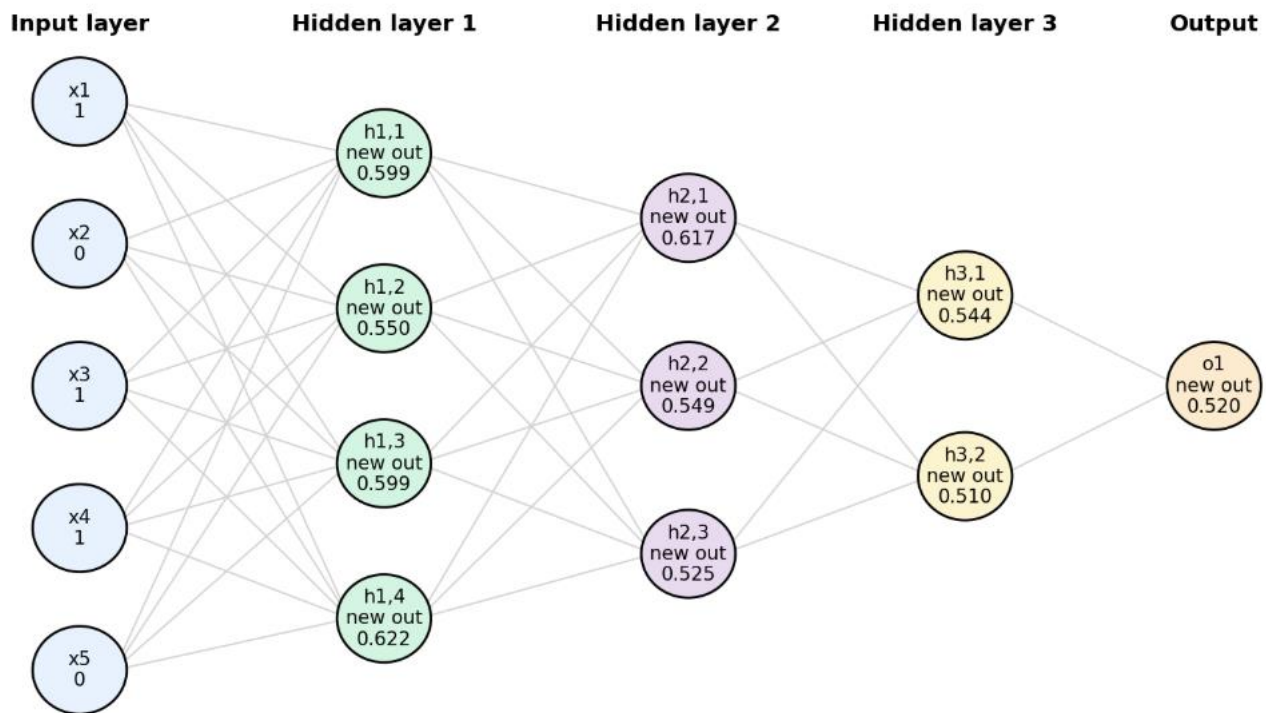
| Value        | Old     | Change  | New     | Note |
|--------------|---------|---------|---------|------|
| w(h1,1→h2,1) | 0.3000  | +0.0001 | 0.3001  |      |
| w(h1,1→h2,2) | -0.2000 | -0.0000 | -0.2000 |      |
| w(h1,1→h2,3) | 0.1000  | -0.0000 | 0.1000  |      |
| w(h1,2→h2,1) | 0.2000  | +0.0000 | 0.2000  |      |
| w(h1,2→h2,2) | 0.4000  | -0.0000 | 0.4000  |      |
| w(h1,2→h2,3) | -0.3000 | -0.0000 | -0.3000 |      |
| w(h1,3→h2,1) | -0.1000 | +0.0001 | -0.0999 |      |
| w(h1,3→h2,2) | 0.1000  | -0.0000 | 0.1000  |      |
| w(h1,3→h2,3) | 0.2000  | -0.0000 | 0.2000  |      |
| w(h1,4→h2,1) | 0.4000  | +0.0001 | 0.4001  |      |
| w(h1,4→h2,2) | -0.1000 | -0.0000 | -0.1000 |      |
| w(h1,4→h2,3) | 0.3000  | -0.0000 | 0.3000  |      |
| b(h2,1)      | 0.0000  | +0.0001 | 0.0001  | bias |
| b(h2,2)      | 0.1000  | -0.0000 | 0.1000  | bias |
| b(h2,3)      | -0.1000 | -0.0001 | -0.1001 | bias |

### Updates: Input → Hidden Layer 1

| Value      | Old     | Change  | New     | Note |
|------------|---------|---------|---------|------|
| w(x1→h1,1) | 0.2000  | +0.0000 | 0.2000  |      |
| w(x1→h1,2) | -0.1000 | +0.0000 | -0.1000 |      |
| w(x1→h1,3) | 0.4000  | -0.0000 | 0.4000  |      |
| w(x1→h1,4) | 0.1000  | +0.0000 | 0.1000  |      |
| w(x2→h1,1) | 0.0000  | +0.0000 | 0.0000  |      |
| w(x2→h1,2) | 0.3000  | +0.0000 | 0.3000  |      |
| w(x2→h1,3) | -0.2000 | -0.0000 | -0.2000 |      |
| w(x2→h1,4) | 0.2000  | +0.0000 | 0.2000  |      |
| w(x3→h1,1) | -0.3000 | +0.0000 | -0.3000 |      |
| w(x3→h1,2) | 0.2000  | +0.0000 | 0.2000  |      |
| w(x3→h1,3) | 0.1000  | -0.0000 | 0.1000  |      |
| w(x3→h1,4) | -0.1000 | +0.0000 | -0.1000 |      |
| w(x4→h1,1) | 0.4000  | +0.0000 | 0.4000  |      |
| w(x4→h1,2) | 0.1000  | +0.0000 | 0.1000  |      |
| w(x4→h1,3) | 0.0000  | -0.0000 | -0.0000 |      |
| w(x4→h1,4) | 0.3000  | +0.0000 | 0.3000  |      |
| w(x5→h1,1) | 0.1000  | +0.0000 | 0.1000  |      |
| w(x5→h1,2) | -0.2000 | +0.0000 | -0.2000 |      |
| w(x5→h1,3) | 0.3000  | -0.0000 | 0.3000  |      |
| w(x5→h1,4) | 0.2000  | +0.0000 | 0.2000  |      |
| b(h1,1)    | 0.1000  | +0.0000 | 0.1000  | bias |
| b(h1,2)    | 0.0000  | +0.0000 | 0.0000  | bias |

|         |         |         |         |      |
|---------|---------|---------|---------|------|
| b(h1,3) | -0.1000 | -0.0000 | -0.1000 | bias |
| b(h1,4) | 0.2000  | +0.0000 | 0.2000  | bias |

### Step 6: After Updating Weights, Next Feed Forward Check



After one update: output improves from 0.5152 to 0.5200; loss changes from 0.1175 to 0.1152

## 6. Final Summary

- Initial output after feed forward = 0.5152
- Initial raw error = 0.4848
- Initial squared error/loss = 0.1175
- Output delta  $\delta_{o1}$  = 0.1211
- After one update, next feed-forward output = 0.5200
- After one update, new loss = 0.1152
- Training continues by repeating: Feed Forward  $\rightarrow$  Error  $\rightarrow$  Backpropagation  $\rightarrow$  Update Weights.

### Question 3: XOR Gate Using Multi-Layer Perceptron

XOR is not linearly separable, so a single perceptron cannot solve it. A small MLP with one hidden layer can learn XOR.

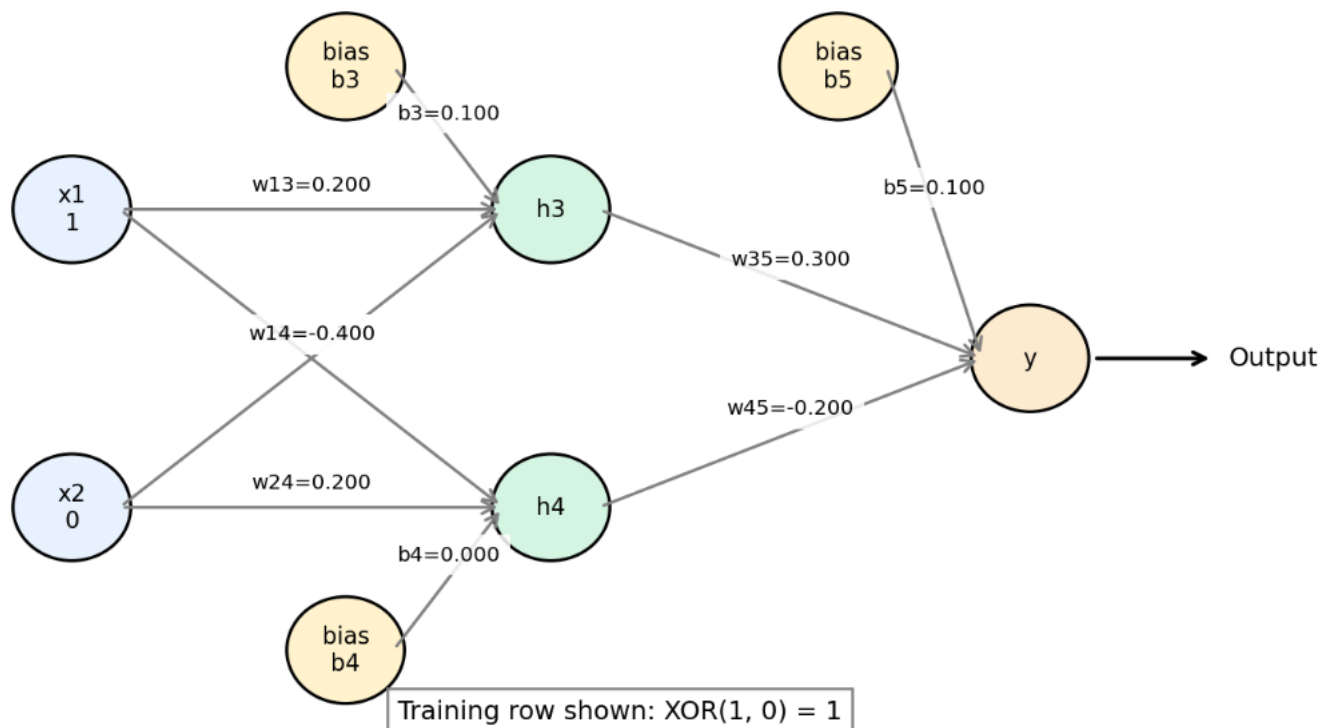
Chosen structure: 2 inputs → 2 hidden neurons → 1 output neuron.

Worked training row:  $x_1 = 1, x_2 = 0, \text{target} = 1$ . Learning rate  $\eta = 0.5$ . Activation =  $\text{sigmoid}(\text{net}) = 1 / (1 + e^{(-\text{net})})$ .

#### 1. XOR Truth Table

| x1 | x2 | XOR target |
|----|----|------------|
| 0  | 0  | 0          |
| 0  | 1  | 1          |
| 1  | 0  | 1          |
| 1  | 1  | 0          |

#### Step 0: XOR MLP Initial Structure



#### 2. Given Initial Weights and Biases

| Connection | Weight | Connection | Weight |
|------------|--------|------------|--------|
| w13        | 0.20   | w23        | -0.30  |
| w14        | -0.40  | w24        | 0.20   |
| w35        | 0.30   | w45        | -0.20  |



|    |      |    |      |
|----|------|----|------|
| b3 | 0.10 | b4 | 0.00 |
| b5 | 0.10 |    |      |

### 3. Feed Forward Pass

#### 3.1 Hidden Layer

$$\text{net3} = x_1w_{13} + x_2w_{23} + b_3$$

$$\text{net3} = 1(0.20) + 0(-0.30) + 0.10 = 0.3000$$

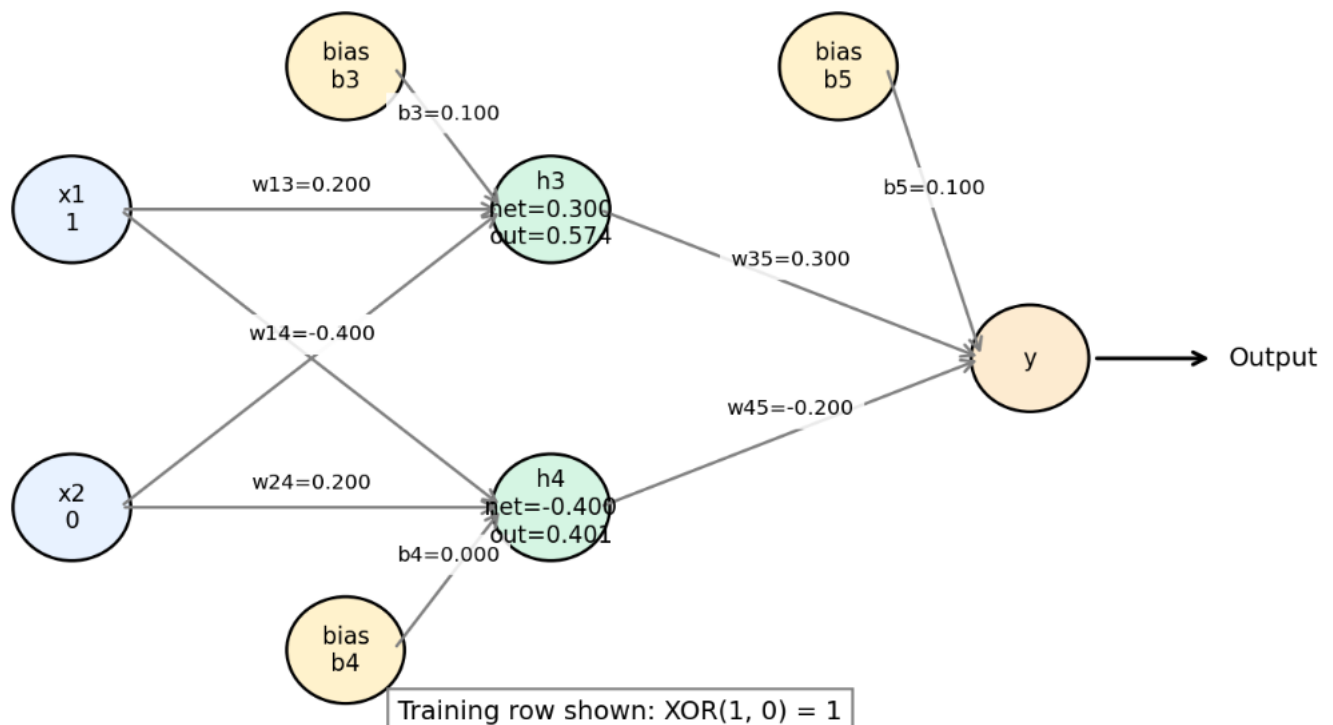
$$h_3 = \text{sigmoid}(0.3000) = 0.5744$$

$$\text{net4} = x_1w_{14} + x_2w_{24} + b_4$$

$$\text{net4} = 1(-0.40) + 0(0.20) + 0.00 = -0.4000$$

$$h_4 = \text{sigmoid}(-0.4000) = 0.4013$$

#### Step 1: Hidden Neurons Calculated



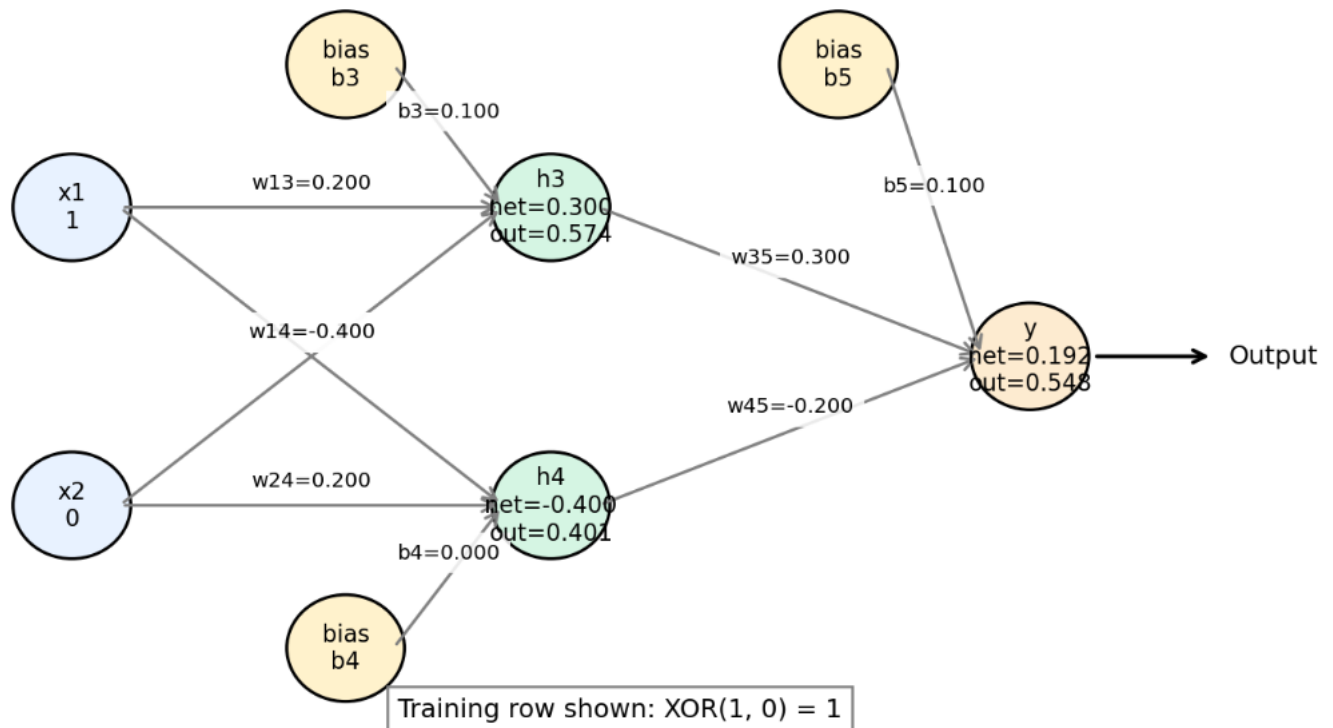
#### 3.2 Output Layer

$$\text{net5} = h_3w_{35} + h_4w_{45} + b_5$$

$$\text{net5} = 0.5744(0.30) + 0.4013(-0.20) + 0.10 = 0.1921$$

$$\text{actual output } y = \text{sigmoid}(0.1921) = 0.5479$$

## Step 2: Output Neuron Calculated



## 4. Error Calculation

Raw error = target - actual = 1 - 0.5479 = 0.4521

Squared error/loss =  $1/2(\text{target} - \text{actual})^2 = 1/2(0.4521)^2 = 0.1022$

## 5. Backpropagation

Output delta:  $\delta_5 = y(1 - y)(\text{target} - y)$

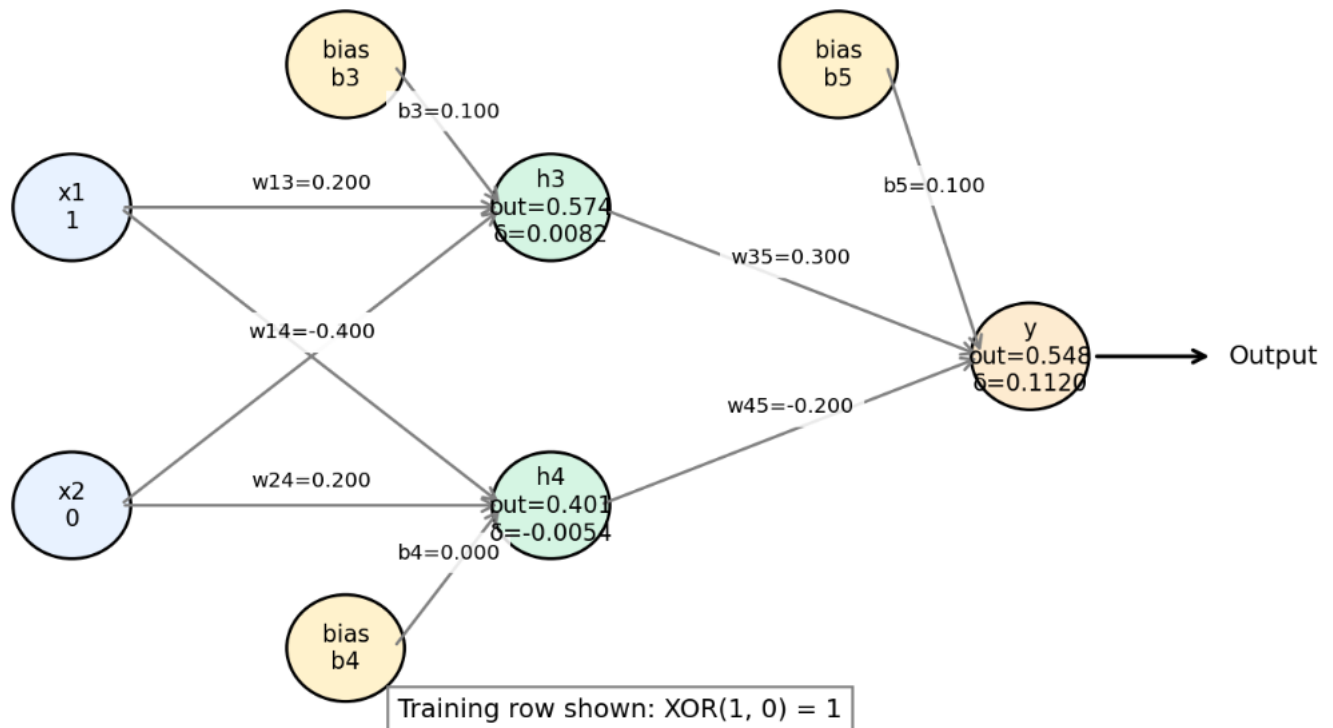
$\delta_5 = 0.5479(1 - 0.5479)(1 - 0.5479) = 0.1120$

Hidden delta:  $\delta_{\text{hidden}} = \text{hidden\_output}(1 - \text{hidden\_output})(\delta_{\text{next}} \times \text{connecting\_weight})$

$\delta_3 = 0.5744(1 - 0.5744)(0.1120 \times 0.30) = 0.0082$

$\delta_4 = 0.4013(1 - 0.4013)(0.1120 \times -0.20) = -0.0054$

### Step 3: Backpropagation Deltas Calculated

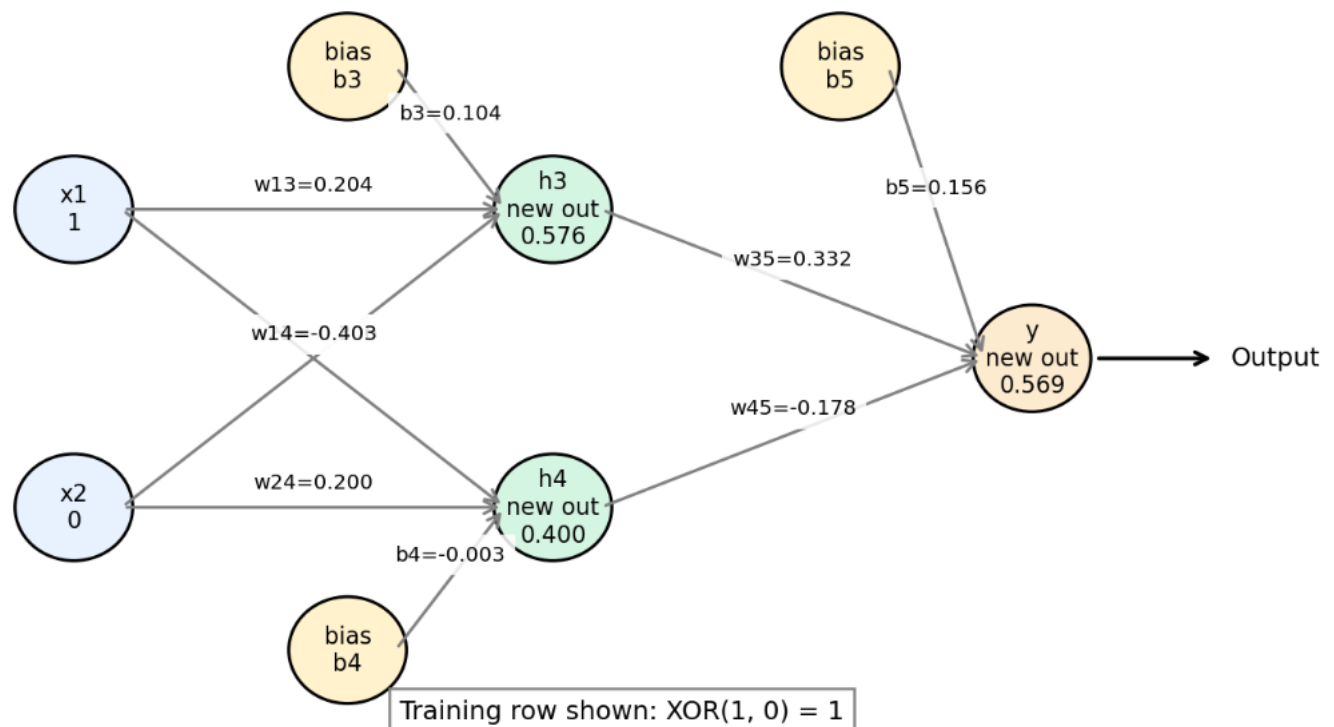


## 6. Update Weights and Biases

Formula: new weight = old weight +  $\eta \times \delta \times \text{input}$ . Bias formula: new bias = old bias +  $\eta \times \delta$ .

| Value | Old     | Change  | New     |
|-------|---------|---------|---------|
| w35   | 0.3000  | +0.0322 | 0.3322  |
| w45   | -0.2000 | +0.0225 | -0.1775 |
| b5    | 0.1000  | +0.0560 | 0.1560  |
| w13   | 0.2000  | +0.0041 | 0.2041  |
| w23   | -0.3000 | +0.0000 | -0.3000 |
| b3    | 0.1000  | +0.0041 | 0.1041  |
| w14   | -0.4000 | -0.0027 | -0.4027 |
| w24   | 0.2000  | -0.0000 | 0.2000  |
| b4    | 0.0000  | -0.0027 | -0.0027 |

## Step 4: After Updating Weights and Checking Again



## 7. Check After One Update

After one weight update, running feed forward again gives new output = 0.5687.

Loss changes from 0.1022 to 0.0930. Training continues by repeating feed forward  $\rightarrow$  error  $\rightarrow$  backpropagation  $\rightarrow$  update weights for all XOR rows.

## 8. How MLP Solves XOR Conceptually

A hidden layer allows the MLP to create intermediate features. One hidden neuron can learn one boundary and the second hidden neuron can learn another boundary. The output neuron then combines these hidden features to produce XOR.

The following table uses a separate already-trained illustrative set of large weights to show that a 2-hidden-neuron MLP can approximate the XOR truth table.

| $x_1$ | $x_2$ | Target XOR | MLP output | Rounded output |
|-------|-------|------------|------------|----------------|
| 0     | 0     | 0          | 0.0000     | 0              |
| 0     | 1     | 1          | 1.0000     | 1              |
| 1     | 0     | 1          | 1.0000     | 1              |
| 1     | 1     | 0          | 0.0000     | 0              |

## Final Summary

- XOR needs an MLP because XOR is not solved by a single straight-line decision boundary.
- For training row XOR(1,0), first output = 0.5479.
- Raw error = 0.4521 and squared loss = 0.1022.
- Deltas are  $\delta_5 = 0.1120$ ,  $\delta_3 = 0.0082$ ,  $\delta_4 = -0.0054$ .
- After one update, output becomes 0.5687.
- To fully train XOR, repeat the same process for all four truth table rows over many epochs.

## Question 4: XNOR Gate Using Multi-Layer Perceptron

XNOR gives output 1 when both inputs are the same. Like XOR, XNOR is not linearly separable, so a single perceptron cannot solve it properly. A small MLP with a hidden layer can learn XNOR.

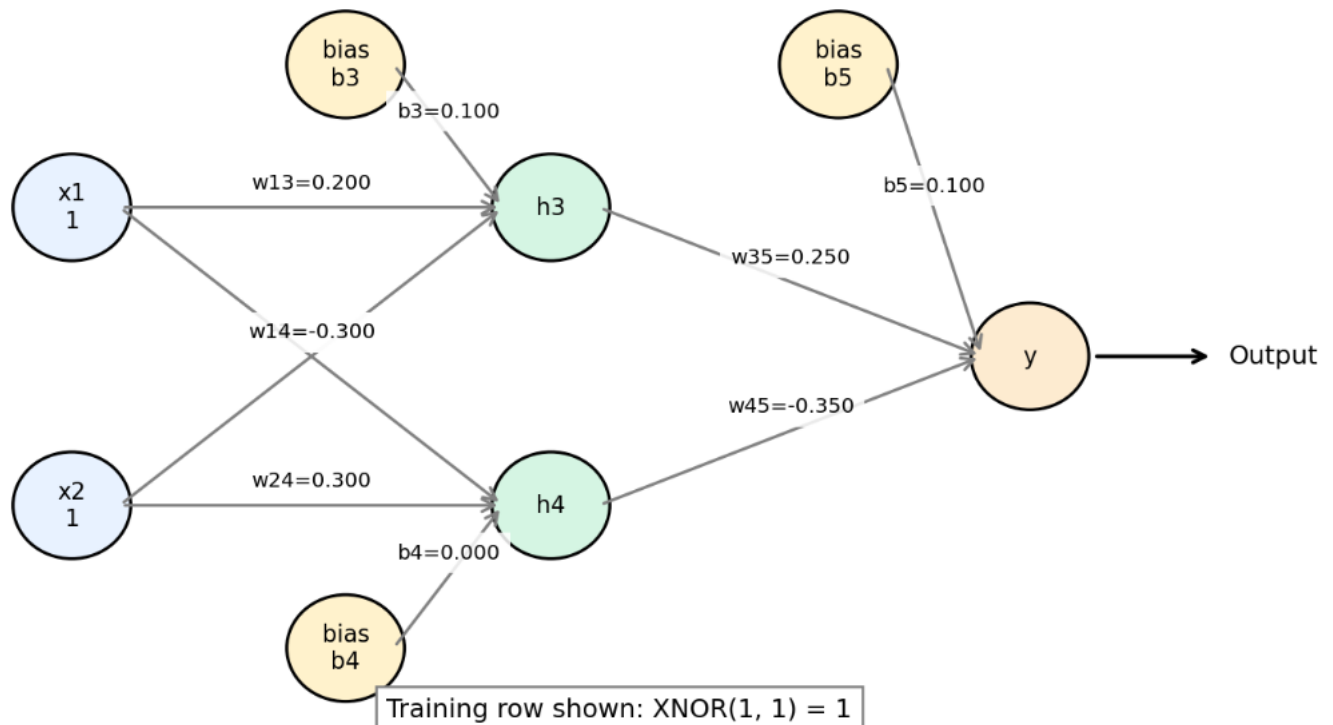
Chosen structure: 2 inputs  $\rightarrow$  2 hidden neurons  $\rightarrow$  1 output neuron.

Worked training row:  $x_1 = 1$ ,  $x_2 = 1$ , target = 1. Learning rate  $\eta = 0.5$ . Activation =  $\text{sigmoid}(\text{net}) = 1 / (1 + e^{(-\text{net})})$ .

### 1. XNOR Truth Table

| x1 | x2 | XNOR target |
|----|----|-------------|
| 0  | 0  | 1           |
| 0  | 1  | 0           |
| 1  | 0  | 0           |
| 1  | 1  | 1           |

## Step 0: XNOR MLP Initial Structure



## 2. Given Initial Weights and Biases

| Connection | Weight | Connection | Weight |
|------------|--------|------------|--------|
| w13        | 0.20   | w23        | -0.10  |
| w14        | -0.30  | w24        | 0.30   |
| w35        | 0.25   | w45        | -0.35  |
| b3         | 0.10   | b4         | 0.00   |
| b5         | 0.10   |            |        |

## 3. Feed Forward Pass

### 3.1 Hidden Layer

$$\text{net3} = x1w13 + x2w23 + b3$$

$$\text{net3} = 1(0.20) + 1(-0.10) + 0.10 = 0.2000$$

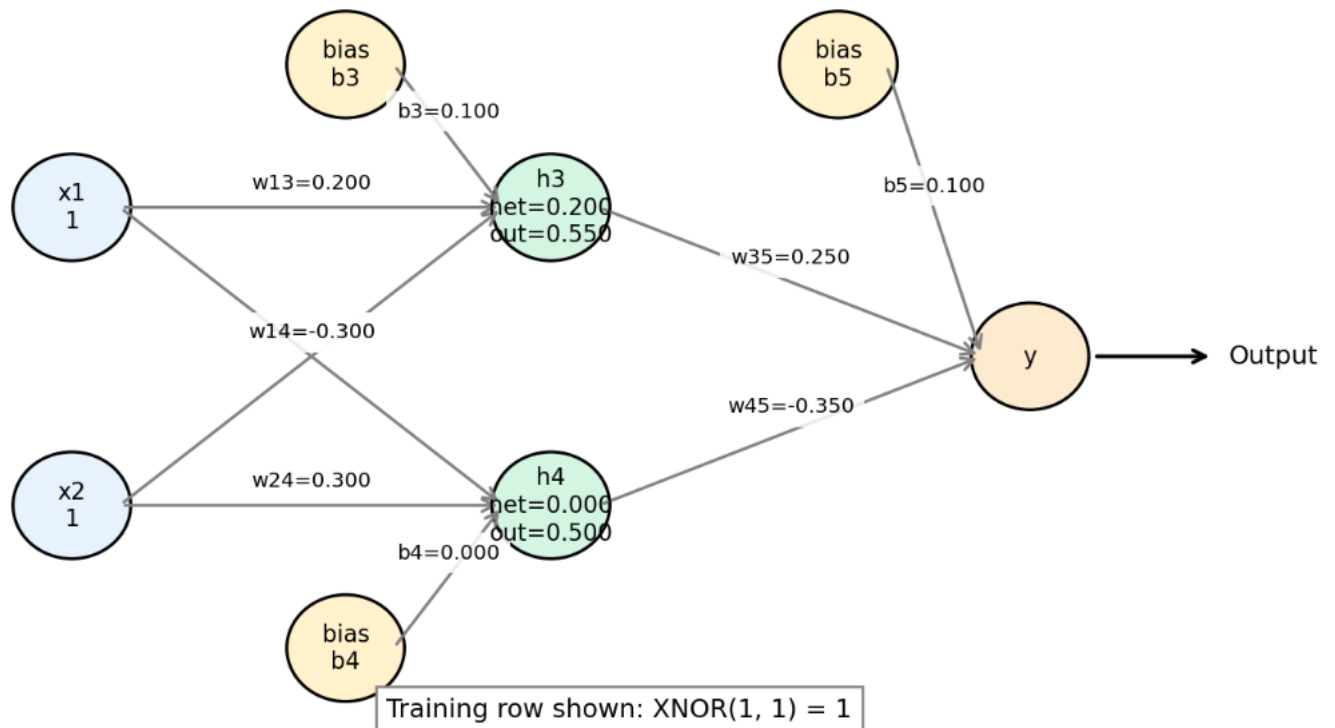
$$h3 = \text{sigmoid}(0.2000) = 0.5498$$

$$\text{net4} = x1w14 + x2w24 + b4$$

$$\text{net4} = 1(-0.30) + 1(0.30) + 0.00 = 0.0000$$

$$h4 = \text{sigmoid}(0.0000) = 0.5000$$

## Step 1: Hidden Neurons Calculated



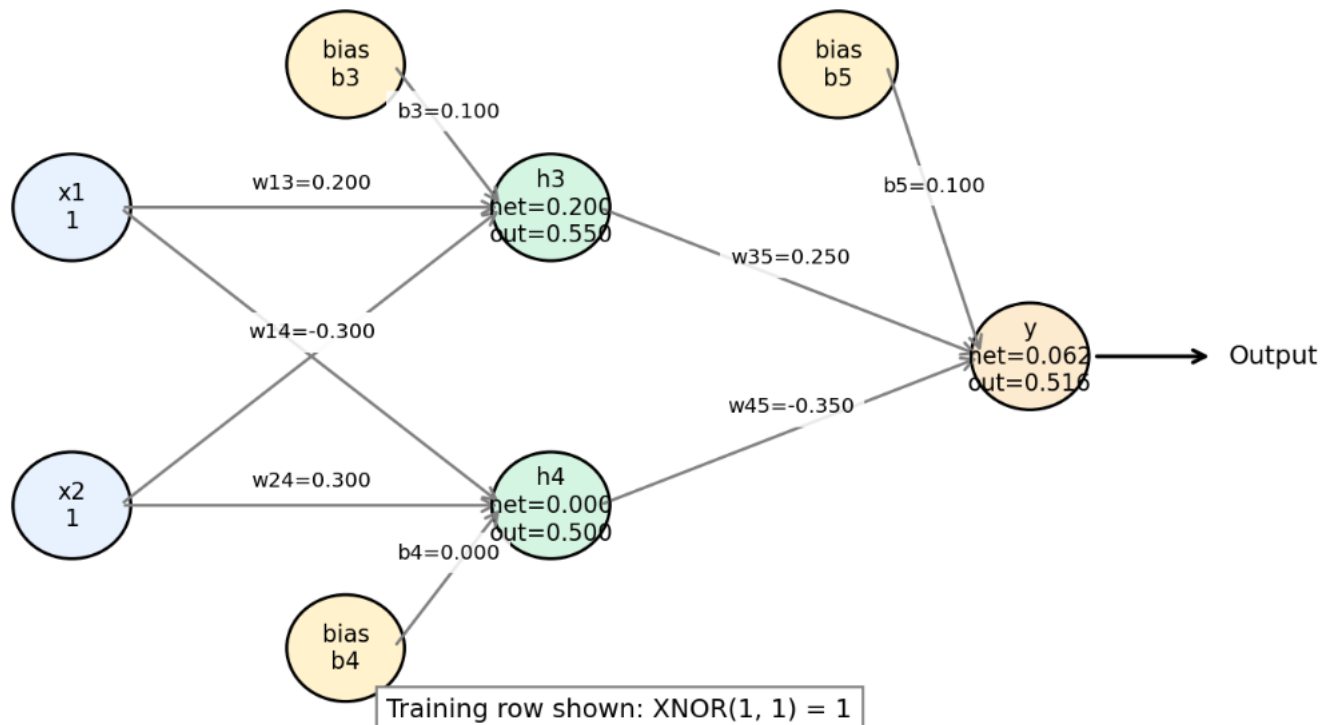
### 3.2 Output Layer

$$\text{net}_5 = h_3w_{35} + h_4w_{45} + b_5$$

$$\text{net}_5 = 0.5498(0.25) + 0.5000(-0.35) + 0.10 = 0.0625$$

$$\text{actual output } y = \text{sigmoid}(0.0625) = 0.5156$$

## Step 2: Output Neuron Calculated



## 4. Error Calculation

Raw error = target - actual = 1 - 0.5156 = 0.4844

Squared error/loss =  $1/2(\text{target} - \text{actual})^2 = 1/2(0.4844)^2 = 0.1173$

## 5. Backpropagation

Output delta:  $\delta_5 = y(1 - y)(\text{target} - y)$

$\delta_5 = 0.5156(1 - 0.5156)(1 - 0.5156) = 0.1210$

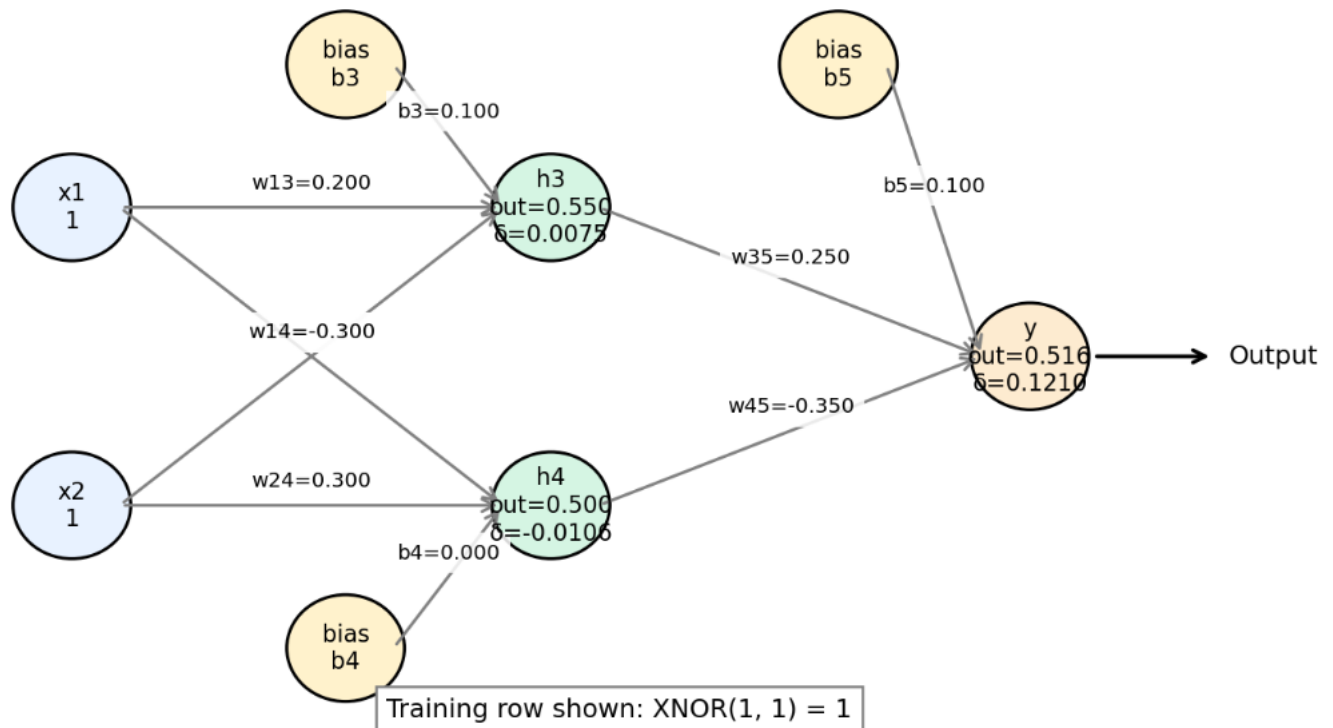
Hidden delta:  $\delta_{\text{hidden}} = \text{hidden\_output}(1 - \text{hidden\_output})(\delta_{\text{next}} \times \text{connecting\_weight})$

$\delta_3 = 0.5498(1 - 0.5498)(0.1210 \times 0.25) = 0.0075$

$\delta_4 = 0.5000(1 - 0.5000)(0.1210 \times -0.35) = -0.0106$



### Step 3: Backpropagation Deltas Calculated

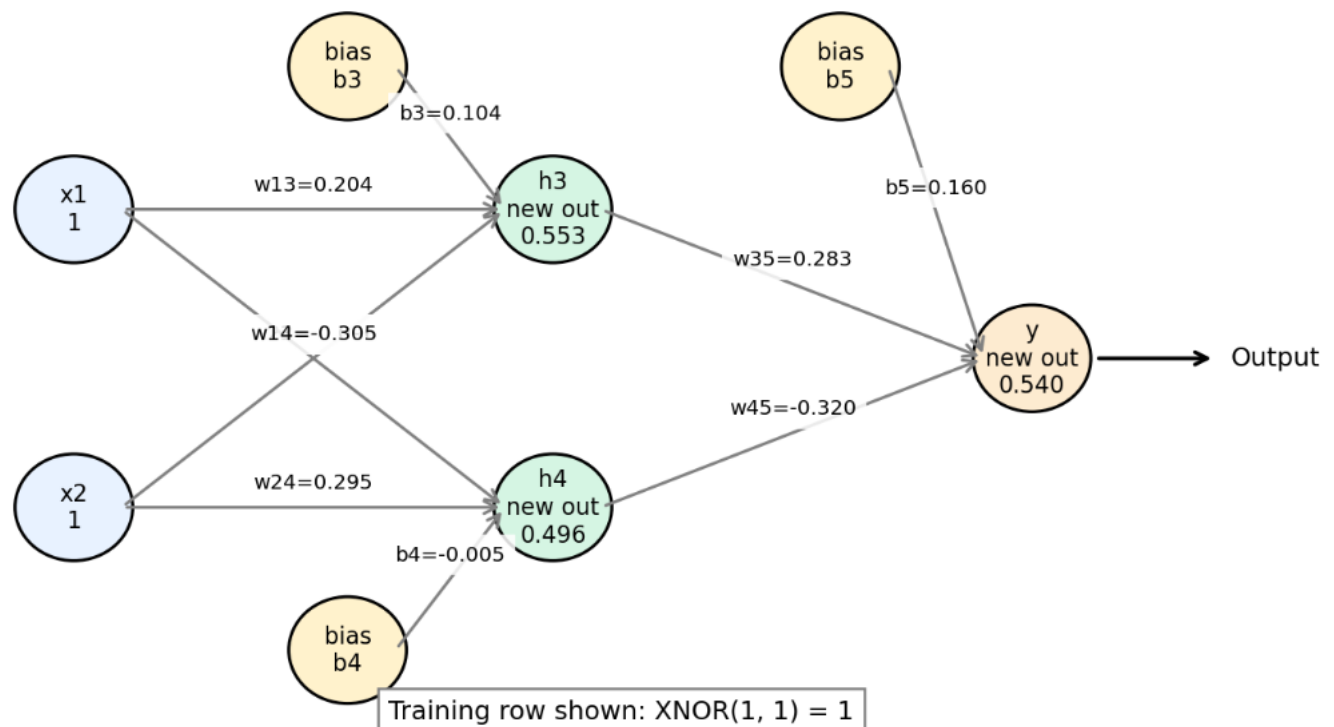


## 6. Update Weights and Biases

Formula: new weight = old weight +  $\eta \times \delta \times \text{input}$ . Bias formula: new bias = old bias +  $\eta \times \delta$ .

| Value | Old     | Change  | New     |
|-------|---------|---------|---------|
| w35   | 0.2500  | +0.0333 | 0.2833  |
| w45   | -0.3500 | +0.0302 | -0.3198 |
| b5    | 0.1000  | +0.0605 | 0.1605  |
| w13   | 0.2000  | +0.0037 | 0.2037  |
| w23   | -0.1000 | +0.0037 | -0.0963 |
| b3    | 0.1000  | +0.0037 | 0.1037  |
| w14   | -0.3000 | -0.0053 | -0.3053 |
| w24   | 0.3000  | -0.0053 | 0.2947  |
| b4    | 0.0000  | -0.0053 | -0.0053 |

## Step 4: After Updating Weights and Checking Again



## 7. Check After One Update

After one weight update, running feed forward again gives new output = 0.5395.

Loss changes from 0.1173 to 0.1060. Training continues by repeating feed forward  $\rightarrow$  error  $\rightarrow$  backpropagation  $\rightarrow$  update weights for all XNOR rows.

## 8. How MLP Solves XNOR Conceptually

A hidden layer lets the MLP create intermediate logical features. One hidden neuron can learn the AND-like case and another hidden neuron can learn the NOR-like case. The output neuron then combines those hidden features to produce XNOR.

The following table uses a separate already-trained illustrative set of large weights to show that a 2-hidden-neuron MLP can approximate the XNOR truth table.

| $x_1$ | $x_2$ | Target XNOR | MLP output | Rounded output |
|-------|-------|-------------|------------|----------------|
| 0     | 0     | 1           | 1.0000     | 1              |
| 0     | 1     | 0           | 0.0000     | 0              |
| 1     | 0     | 0           | 0.0000     | 0              |
| 1     | 1     | 1           | 1.0000     | 1              |

## Final Summary

- XNOR gives 1 when both inputs are the same: 00 or 11.
- XNOR needs an MLP because it is not solved well by a single straight-line decision boundary.
- For training row XNOR(1,1), first output = 0.5156.
- Raw error = 0.4844 and squared loss = 0.1173.
- Deltas are  $\delta_5 = 0.1210$ ,  $\delta_3 = 0.0075$ ,  $\delta_4 = -0.0106$ .
- After one update, output becomes 0.5395.
- To fully train XNOR, repeat the same process for all four truth table rows over many epochs.

---

Complete

---